	high-level mem optimizations	int-to-ptr casts	low-level ptr manipulation	all integer optimizations	ptr arith is pure
Flat		✓	✓	✓	✓
Memarian et al.	✓	✓			✓
CompCert	✓			✓	✓
Besson et al.	✓	✓			✓
Kang et al.	✓	†	✓	✓	
Ours	✓	✓	✓	✓	✓

Table 2. Comparison between different memory models: whether they enable high-level memory optimizations (such as store forwarding), whether they support integer-to-pointer casts, whether they support languages with low-level pointer manipulation (such as doing XORs of pointers), whether they support all standard integer optimizations (such as equality propagation and reassociation), and whether pointer arithmetic and cast operations are pure (since otherwise they constrain movement of such instructions). † does not support cast of $p + n$.

because observing the underlying address of a pointer is not allowed. For example, using a pointer value as a hash key is undefined in their model.

Kang et al. [2015] proposed a low-level memory model that supports pointer-to-integer casts. However, the model does not support casting a pointer one-past-the-end of an object to an integer and back: this round-trip resulted in **poison** in their semantics, while it is well-defined in C. Moreover, in their semantics, `ptrtoint` is an effectful operation that cannot be removed even if the result is not used, and `inttoptr` is not freely movable across `malloc` and `free`. Our work fixes both of these problems by combining logical and physical pointers in one memory model.

Table 2 shows a summary comparing the different memory models and ours.

Lee et al. [2017] proposed adjusting the semantics of **poison** and **undef** in order to fix some problems in LLVM. They do not address problems with pointer values and thus their work is orthogonal to ours, and in fact we reuse their value semantics for our work.

Both Alive [Lopes et al. 2015] (an automatic peephole optimization verifier) and Souper [Sasnauskas et al. 2017] (a superoptimizer) encode the semantics of LLVM IR into SMT. Souper currently does not support memory operations, and Alive does not support integer-to-pointer casts. Both could be extended with our new memory model in order to support a larger set of transformations.

Verification tools for C/C++ programs often use memory models that try to split objects into different heaps to speedup verification time (e.g., [Gurfinkel and Navas 2017; Wang et al. 2017]). However, usually these models either do not support integer-to-pointer casts or they fallback to a flat memory model when these casts are used. The trade-offs for IR and verification tools memory models are different, since verification tools can soundly lose precision for uncommon cases, for example, while for compiler IRs it cannot since the IR memory model needs to justify the correctness of transformations in all cases. SMT-based verification have highly-optimized SMT encodings of their memory models, while ours is not trivial to encode efficiently.

8 DISCUSSION AND FUTURE WORK

Type Punning. In our semantics, loading, e.g., from a memory location that contains a pointer at an integer type results in **poison** due to type punning. One may wonder why we did not specify the load to, instead, perform an implicit pointer-to-integer cast. Disallowing implicit pointer casts is necessary to justify load-store elimination:

```
int q, **x = malloc(sizeof(int*));
*x = &q;
int i = *(int*)x // implicit pointer-to-integer cast
*(int*)x = i; // remove; it is redundant
```